PYTHON ACTIVITY-BASED LEARNING

Complete Record of 10 Programs

Includes aim, objectives, algorithm, pseudocode, flowchart, source code and sample output for each program.

Note: The sample outputs are generated from the supplied programs using representative inputs. Console preview images are included separately; they are not genuine screenshots captured from PyCharm.

Program 1: Water Tank Level Monitoring

Aim

Water Tank Level Monitoring

Objective

Monitor water in a tank for three days, keep the level within 0 and 1000 units, and stop when it reaches 800 units.

Algorithm

1. Start; Set capacity=1000, water=500, threshold=800, days=3

2. Repeat for each day from 1 to 3

3. Input inflow and outflow

4. Update water = water + inflow - outflow

5. If water > capacity, display overflow and set water=capacity

6. If water < 0, display shortage and set water=0

7. Display day and water level

8. If water >= threshold, display threshold reached and stop loop

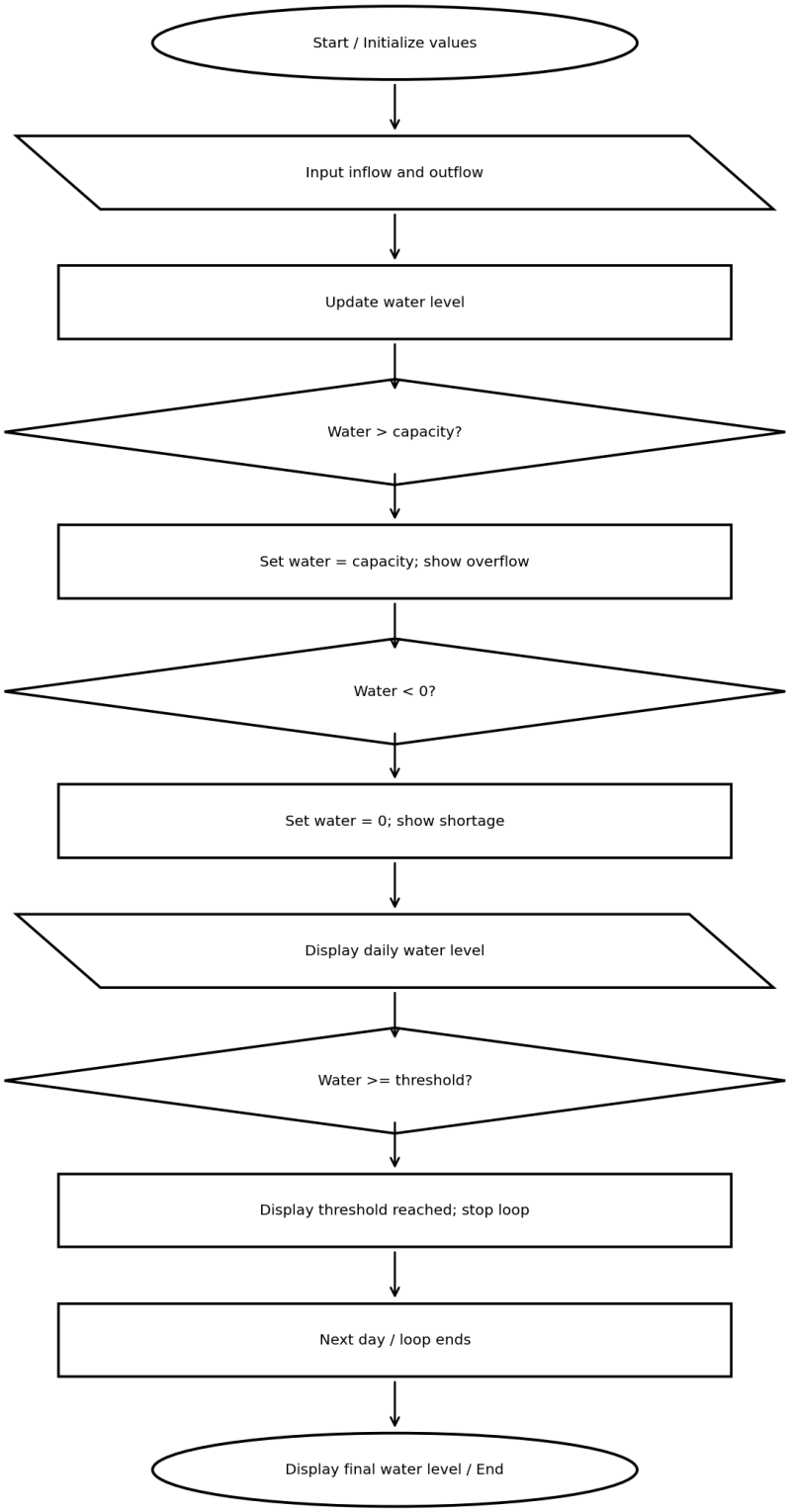
9. After loop, display final water level; End

Pseudocode

```
START
INITIALIZE capacity=1000, water=500, threshold=800, days=3
FOR day = 1 TO days
    INPUT inflow, outflow
    water = water + inflow - outflow
    IF water > capacity THEN
        DISPLAY "Overflow!"
        water = capacity
    END IF
    IF water < 0 THEN
        DISPLAY "Water shortage!"
        water = 0
    END IF
    DISPLAY day, water
    IF water >= threshold THEN
        DISPLAY "Threshold reached"
        EXIT LOOP
    END IF
END FOR
DISPLAY final water
STOP
```

Flowchart

Program 1: Water Tank Level Monitoring



Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision

Python Source Code

```
capacity = 1000
water = 500
threshold = 800
days = 3

for day in range(1, days + 1):
    inflow = int(input("Enter inflow: "))
    outflow = int(input("Enter outflow: "))
```

```
    water = water + inflow - outflow

    if water > capacity:
        print("Overflow!")
        water = capacity

    if water < 0:
        print("Water shortage!")
        water = 0

    print("Day", day, "Water level:", water)

    if water >= threshold:
        print("Threshold reached on day", day)
        break

print("Final water level:", water)
```

Sample Input

```
100
50
200
20
100
30
```

Sample Output

```
Enter inflow: 100
Enter outflow: 50
Day 1 Water level: 550
Enter inflow: 200
Enter outflow: 20
Day 2 Water level: 730
Enter inflow: 100
Enter outflow: 30
Day 3 Water level: 800
Threshold reached on day 3
Final water level: 800
```

Program 2: Shopping Budget Management

Aim

Shopping Budget Management

Objective

Calculate discounted product costs, compare the total with a budget of Rs. 1000, and reduce quantities until the total is within budget.

Algorithm

1. Start; Set budget=1000 and total=0; store product names, prices and quantities

2. For each of the three products, calculate price × quantity

3. If quantity is at least 2, apply 10% discount

4. Display product cost and add it to total

5. Display total cost

6. If total exceeds budget, display budget exceeded

7. For each product, while quantity > 0 and total > budget, subtract one unit price and reduce quantity

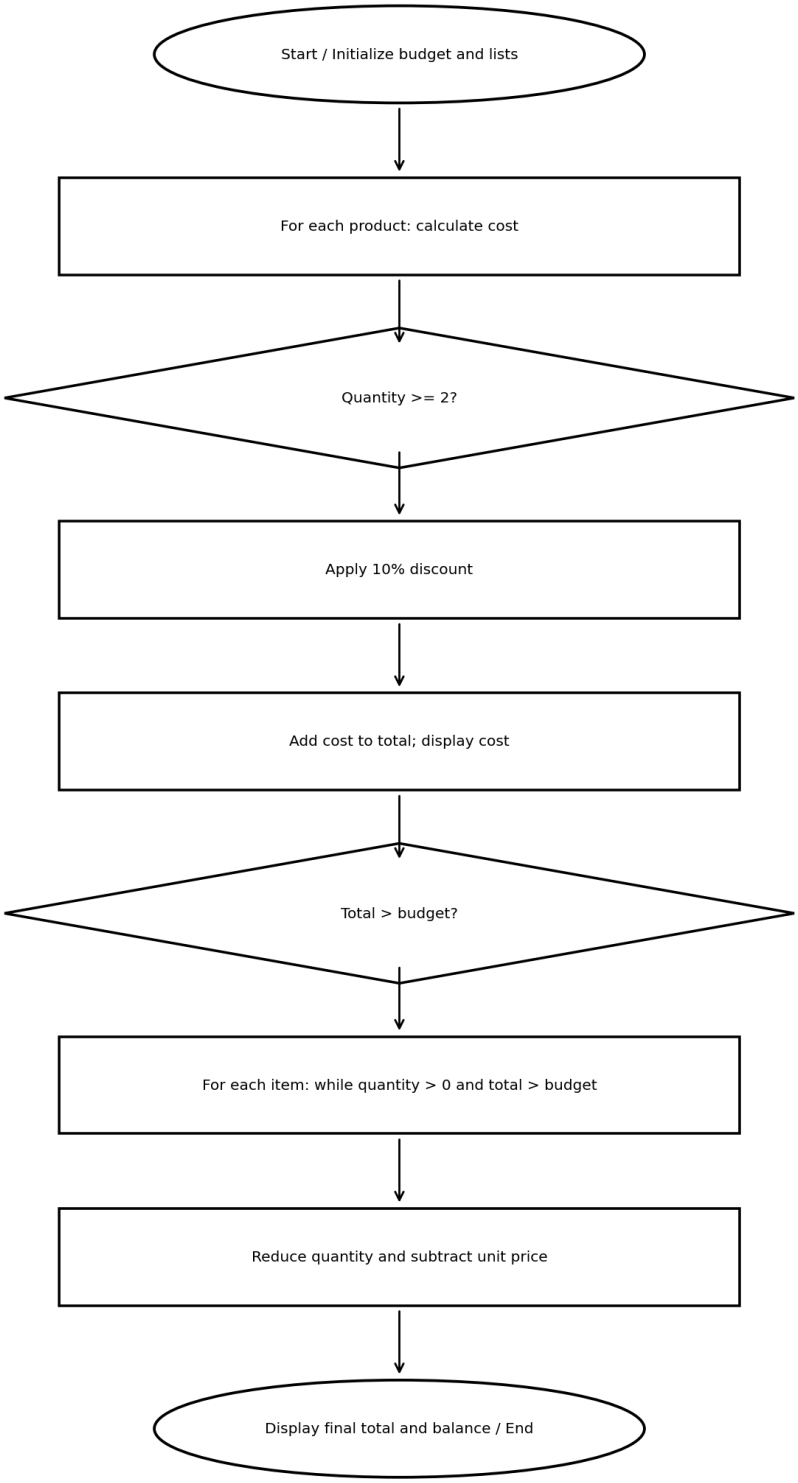
8. Display final total and remaining budget; End

Pseudocode

```
START
SET budget=1000, total=0
STORE products, prices, quantities
FOR each product
    cost = price * quantity
    IF quantity >= 2 THEN
        cost = cost * 0.9
    END IF
    DISPLAY product and cost
    total = total + cost
END FOR
DISPLAY total
IF total > budget THEN
    DISPLAY "Budget exceeded!"
    FOR each product
        WHILE quantity > 0 AND total > budget
            total = total - price
            quantity = quantity - 1
            DISPLAY quantity reduction
        END WHILE
    END FOR
END IF
DISPLAY final total and remaining budget
STOP
```

Flowchart

Program 2: Shopping Budget Management



Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision

Python Source Code

```
budget = 1000
total = 0
products = ["Book", "Pen", "Bag"]
prices = [300, 50, 600]
quantities = [2, 3, 1]

for i in range(3):
    cost = prices[i] * quantities[i]
```

```
    if quantities[i] >= 2:
        cost = cost * 0.9
    print(products[i], "Cost:", cost)
    total = total + cost

print("Total cost:", total)

if total > budget:
    print("Budget exceeded!")
    for i in range(3):
        while quantities[i] > 0 and total > budget:
            total = total - prices[i]
            quantities[i] = quantities[i] - 1
            print("Reduce quantity of", products[i])

print("Final total:", total)
print("Remaining budget:", budget - total)
```

Sample Input

```
No keyboard input (fixed sample data).
```

Sample Output

```
Book Cost: 540.0
Pen Cost: 135.0
Bag Cost: 600
Total cost: 1275.0
Budget exceeded!
Reduce quantity of Book
Final total: 975.0
Remaining budget: 25.0
```

Program 3: Weekly Appliance Electricity Consumption

Aim

Weekly Appliance Electricity Consumption

Objective

Calculate weekly consumption for each appliance, compare it with a 20-unit limit, and calculate the combined consumption for each day.

Algorithm

1. Start; Store appliance names, seven-day consumption data and limit=20

2. For each appliance, calculate sum of its seven daily values

3. Display weekly consumption

4. If weekly total > 20, display limit warning

5. For each day from 1 to 7, set daily_total=0

6. Add that day's consumption for each of the three appliances

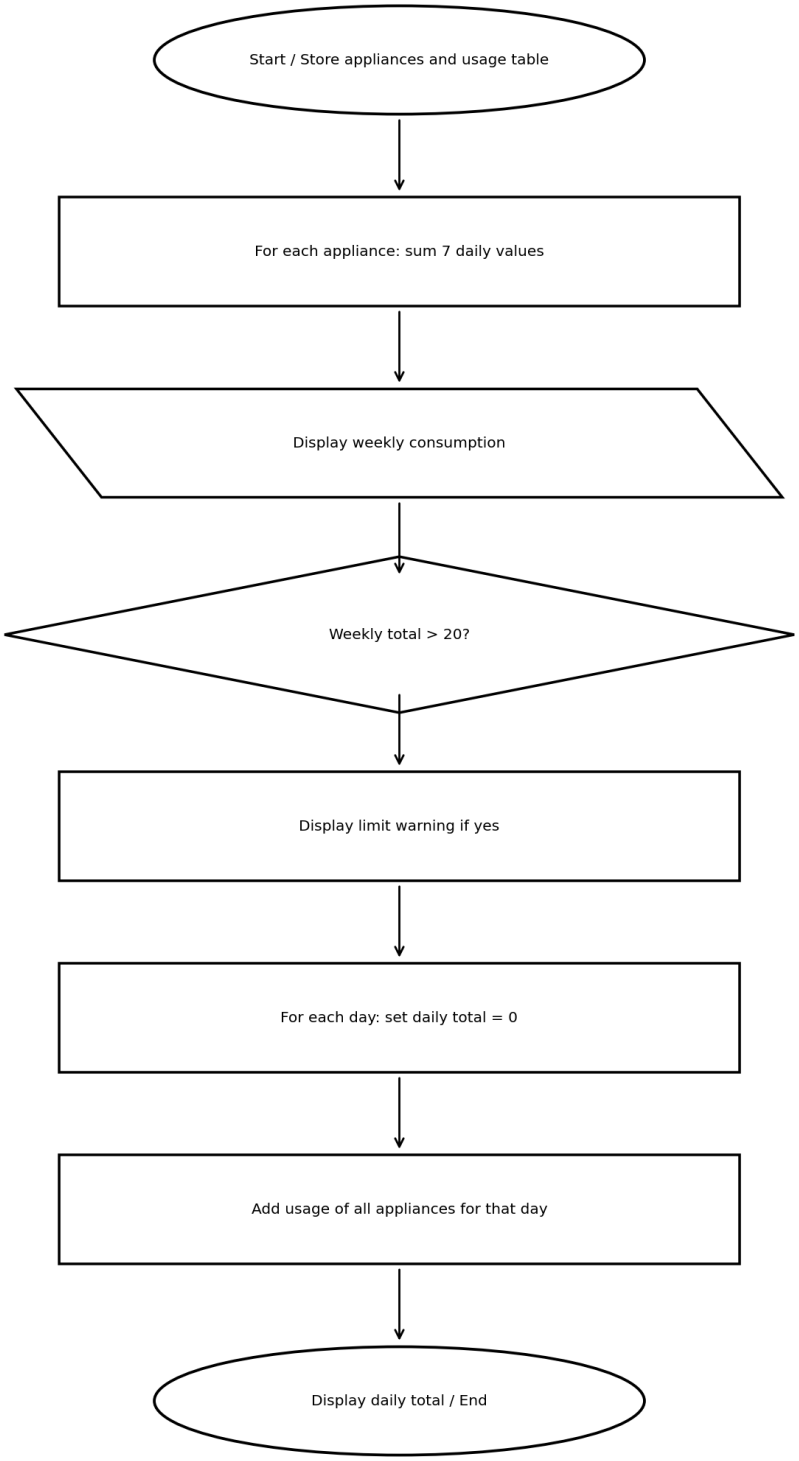
7. Display daily total; End

Pseudocode

```
START
STORE appliance names and 7-day consumption table
SET limit=20
FOR each appliance
    total = SUM(its 7 daily values)
    DISPLAY weekly total
    IF total > limit THEN
        DISPLAY limit warning
    END IF
END FOR
FOR day = 0 TO 6
    daily_total = 0
    FOR each appliance
        daily_total = daily_total + consumption[appliance][day]
    END FOR
    DISPLAY day+1, daily_total
END FOR
STOP
```

Flowchart

Program 3: Weekly Appliance Electricity Consumption



Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision

Python Source Code

```
appliances = ["Fan", "TV", "Fridge"]
consumption = [
    [2, 3, 2, 3, 2, 3, 2],
    [1, 2, 1, 2, 1, 2, 1],
    [4, 4, 4, 4, 4, 4, 4]
]
limit = 20
for i in range(3):
```

```
    total = sum(consumption[i])
    print(appliances[i], "weekly consumption:", total, "units")
    if total > limit:
        print("Consumption exceeds limit!")

for day in range(7):
    daily_total = 0
    for i in range(3):
        daily_total += consumption[i][day]
    print("Day", day + 1, "total:", daily_total, "units")
```

Sample Input

```
No keyboard input (fixed sample data).
```

Sample Output

```
Fan weekly consumption: 17 units
TV weekly consumption: 10 units
Fridge weekly consumption: 28 units
Consumption exceeds limit!
Day 1 total: 7 units
Day 2 total: 9 units
Day 3 total: 7 units
Day 4 total: 9 units
Day 5 total: 7 units
Day 6 total: 9 units
Day 7 total: 7 units
```

Program 4: Bus Passenger Count and Capacity Check

Aim

Bus Passenger Count and Capacity Check

Objective

Track passengers boarding and alighting at four stops, prevent a negative passenger count, and report when the bus exceeds its capacity of 50.

Algorithm

1. Start; Set capacity=50, passengers=0, stops=4

2. Repeat for stops 1 through 4

3. Input boarding and alighting passenger counts

4. Update passengers = passengers + boarding - alighting

5. If passengers < 0, reset passengers to 0

6. Display stop number and passenger count

7. If passengers > 50, display capacity exceeded

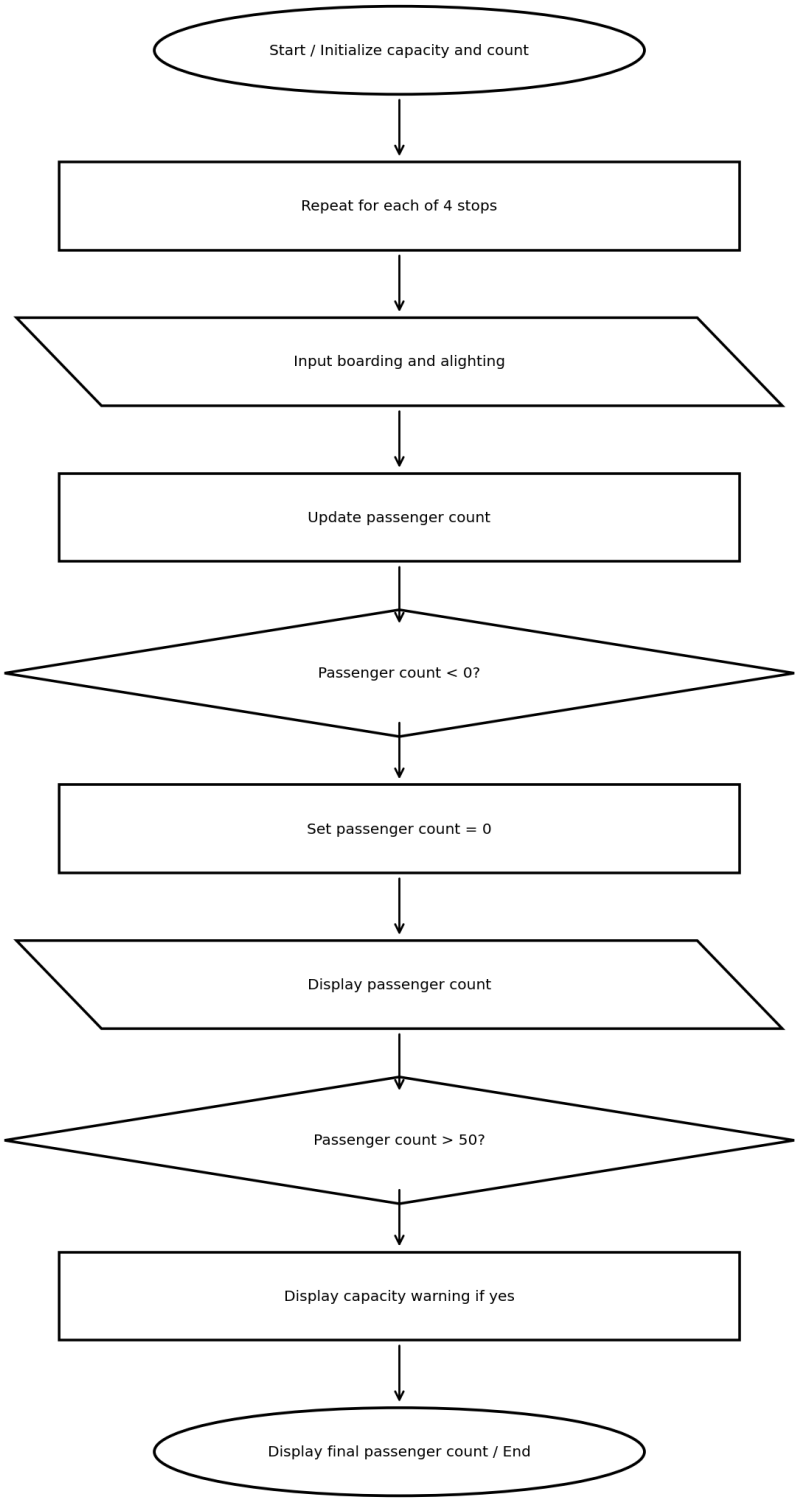
8. After four stops, display final passenger count; End

Pseudocode

```
START
SET capacity=50, passengers=0, stops=4
FOR each stop from 1 to 4
    INPUT boarding, alighting
    passengers = passengers + boarding - alighting
    IF passengers < 0 THEN
        passengers = 0
    END IF
    DISPLAY stop and passengers
    IF passengers > capacity THEN
        DISPLAY "Capacity exceeded!"
    END IF
END FOR
DISPLAY final passengers
STOP
```

Flowchart

Program 4: Bus Passenger Count and Capacity Check



Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision

Python Source Code

```
capacity = 50
passengers = 0
stops = 4

for i in range(1, stops + 1):
    boarding = int(input("Enter boarding passengers: "))
    alighting = int(input("Enter alighting passengers: "))
    passengers = passengers + boarding - alighting
```

```
    if passengers < 0:
        passengers = 0
    print("Stop", i, "Passengers:", passengers)
    if passengers > capacity:
        print("Capacity exceeded!")

print("Final passengers:", passengers)
```

Sample Input

```
20
5
15
3
25
10
5
2
```

Sample Output

```
Enter boarding passengers: 20
Enter alighting passengers: 5
Stop 1 Passengers: 15
Enter boarding passengers: 15
Enter alighting passengers: 3
Stop 2 Passengers: 27
Enter boarding passengers: 25
Enter alighting passengers: 10
Stop 3 Passengers: 42
Enter boarding passengers: 5
Enter alighting passengers: 2
Stop 4 Passengers: 45
Final passengers: 45
```

Program 5: Classroom Seating Arrangement Validation

Aim

Classroom Seating Arrangement Validation

Objective

Validate a seating matrix by checking reserved seats, empty seats, invalid roll numbers, and duplicate roll numbers.

Algorithm

1. Start; Create seating matrix, reserved-seat list, valid-roll list and empty rolls list

2. Set row index i=0; repeat through every row

3. Set column index j=0; repeat through every seat in the row

4. Read roll number at position (i,j)

5. If position is reserved, check whether value is -1; otherwise report reserved-seat error

6. Else if roll=0, report empty seat

7. Else if roll is not in valid list, report invalid roll

8. Else if roll already exists in rolls, report duplicate

9. Otherwise append roll to rolls

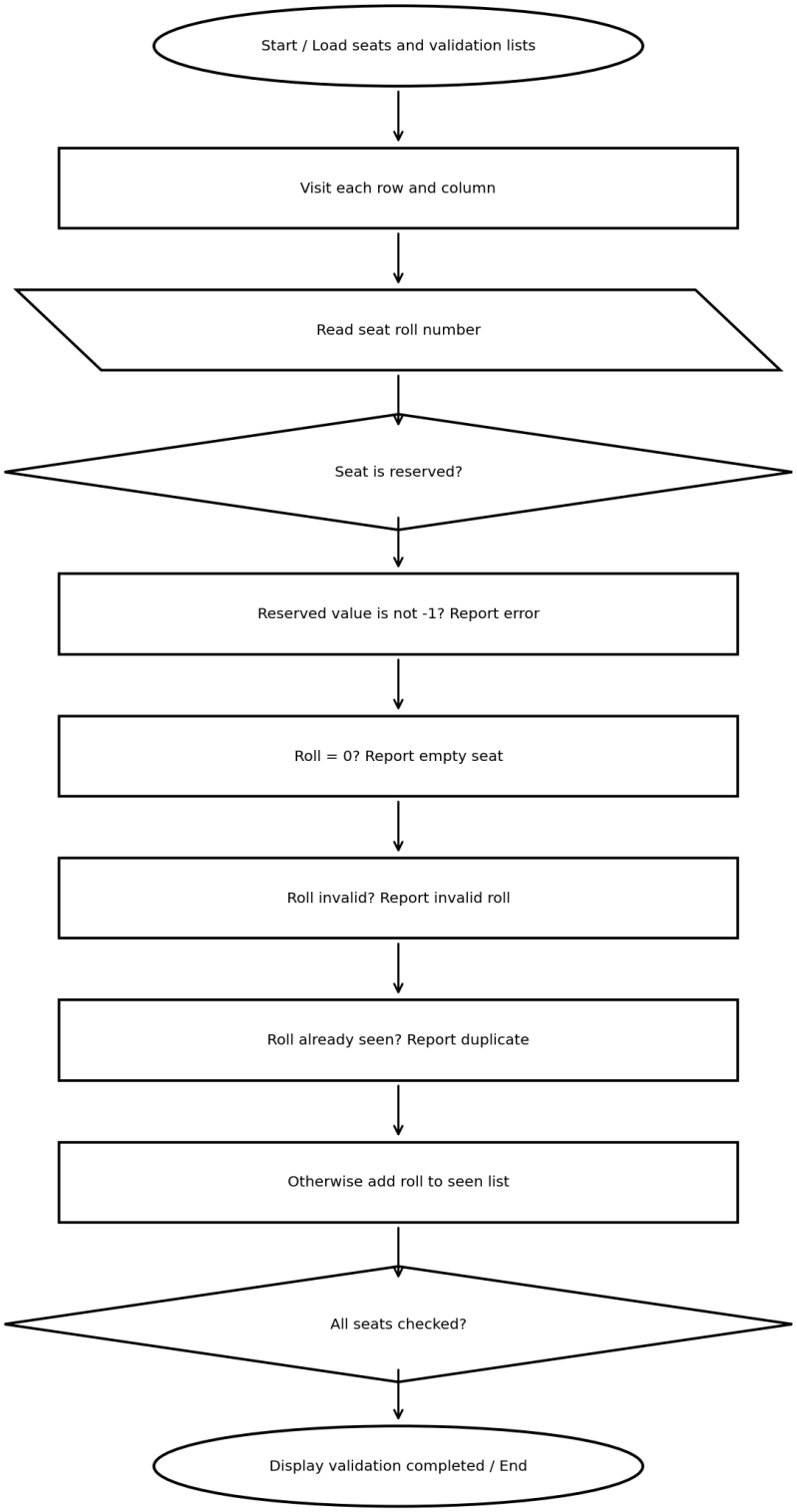
10. Continue until all seats are checked; display completion; End

Pseudocode

```
START
STORE seating matrix, reserved positions and valid roll numbers
SET rolls = empty list
FOR each row i
    FOR each column j
        roll = seats[i][j]
        IF (i,j) is reserved THEN
            IF roll != -1 THEN DISPLAY reserved seat error
        ELSE IF roll == 0 THEN
            DISPLAY empty seat
        ELSE IF roll not in valid_rolls THEN
            DISPLAY invalid roll number
        ELSE IF roll in rolls THEN
            DISPLAY duplicate roll number
        ELSE
            APPEND roll to rolls
        END IF
    END FOR
END FOR
DISPLAY "Seating validation completed."
STOP
```

Flowchart

Program 5: Classroom Seating Arrangement Validation



Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision

Python Source Code

```
seats = [
    [101, 102, 103],
    [104, 0, 106],
    [107, 102, -1]
]
reserved = [(1, 2)]
valid_rolls = [101, 102, 103, 104, 105, 106, 107, 108]
rolls = []
```

```
for i in range(len(seats)):
    for j in range(len(seats[i])):
        roll = seats[i][j]
        if (i, j) in reserved:
            if roll != -1:
                print("Reserved seat error:", i, j)
        elif roll == 0:
            print("Empty seat:", i, j)
        elif roll not in valid_rolls:
            print("Invalid roll number:", roll)
        elif roll in rolls:
            print("Duplicate roll number:", roll)
        else:
            rolls.append(roll)

print("Seating validation completed.")
```

Sample Input

```
No keyboard input (fixed sample data).
```

Sample Output

```
Reserved seat error: 1 2
Empty seat: 1 1
Duplicate roll number: 102
Seating validation completed.
```

Program 6: Student Scholarship Eligibility and Ranking

Aim

Student Scholarship Eligibility and Ranking

Objective

Select students who satisfy the academic, attendance, and family-income conditions, then rank eligible students by aggregate marks and attendance.

Algorithm

1. Start; Store student records; create empty eligible list

2. For each student, read aggregate, subject marks, attendance and income

3. Check aggregate >=80, both subject marks >=60, attendance >=75 and income <=10000

4. If all conditions are true, add student to eligible list

5. Sort eligible students by aggregate and then attendance in descending order

6. Set rank=1; display each eligible student's rank, name, marks and attendance

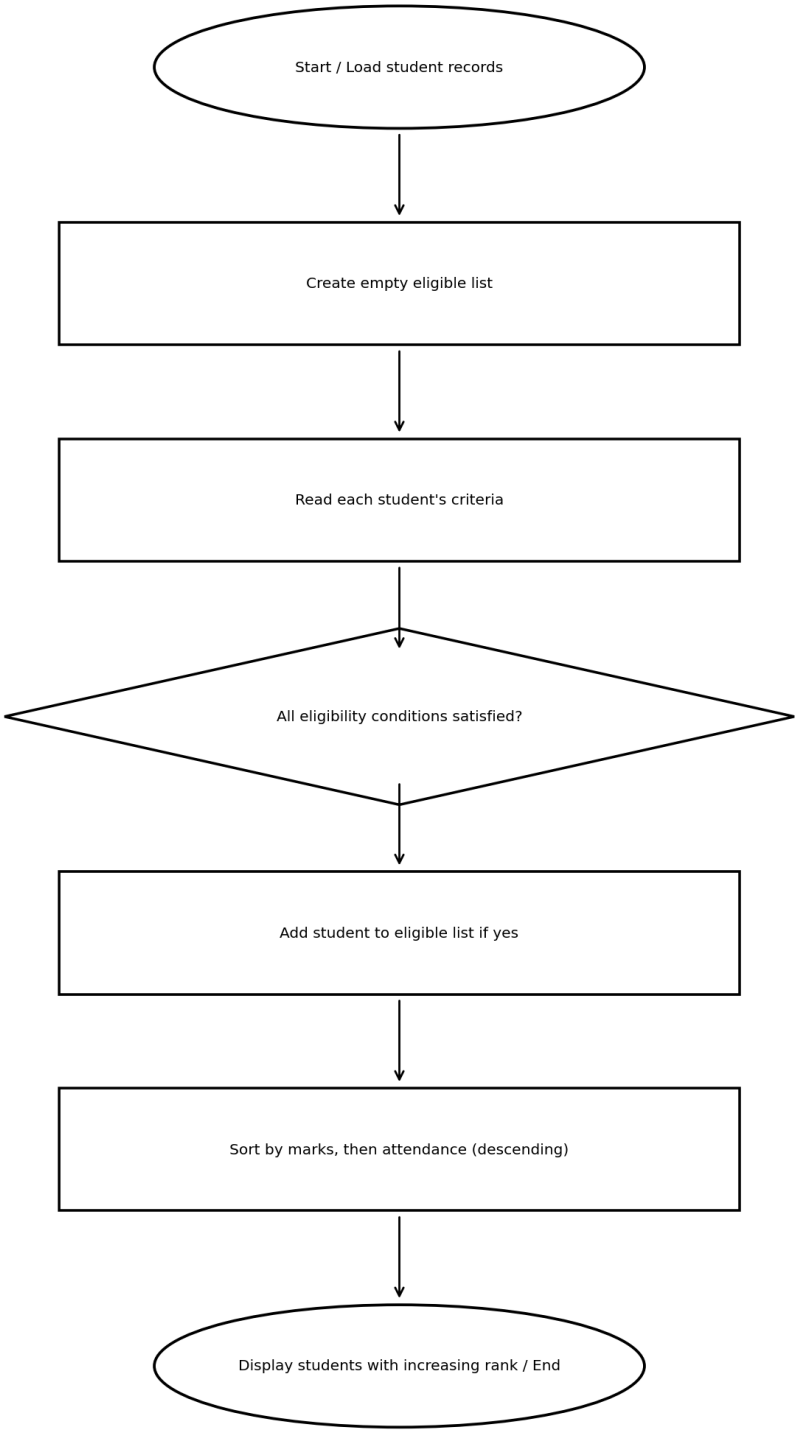
7. Increase rank after each student; End

Pseudocode

```
START
STORE student records
SET eligible = empty list
FOR each student s
    READ aggregate, subject1, subject2, attendance, income
    IF aggregate >= 80 AND subject1 >= 60 AND subject2 >= 60
       AND attendance >= 75 AND income <= 10000 THEN
        ADD s to eligible
    END IF
END FOR
SORT eligible by aggregate, then attendance, descending
SET rank=1
FOR each student in eligible
    DISPLAY rank, name, marks, attendance
    rank = rank + 1
END FOR
STOP
```

Flowchart

Program 6: Student Scholarship Eligibility and Ranking



Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision

Python Source Code

```
students = [
    ["Ravi", 85, 75, 80, 90, 12000],
    ["Raju", 90, 80, 85, 95, 10000],
    ["Sita", 78, 70, 75, 80, 8000],
    ["Kiran", 90, 80, 85, 95, 9000]
]
eligible = []
for s in students:
```

```
    name = s[0]
    aggregate = s[1]
    subject1 = s[2]
    subject2 = s[3]
    attendance = s[4]
    income = s[5]
    if (aggregate >= 80 and subject1 >= 60 and
        subject2 >= 60 and attendance >= 75 and income <= 10000):
        eligible.append(s)

eligible.sort(key=lambda s: (s[1], s[4]), reverse=True)
print("Eligible Students:")
rank = 1
for s in eligible:
    print("Rank:", rank, "Name:", s[0],
          "Marks:", s[1], "Attendance:", s[4])
    rank += 1
```

Sample Input

```
No keyboard input (fixed sample data).
```

Sample Output

```
Eligible Students:
Rank: 1 Name: Raju Marks: 90 Attendance: 95
Rank: 2 Name: Kiran Marks: 90 Attendance: 95
```

Program 7: Employee Salary Statement

Aim

Employee Salary Statement

Objective

Accept an employee's salary components, calculate gross salary, apply a deduction rate based on gross salary, and display net salary.

Algorithm

1. Start; Input employee name, basic salary, allowances, overtime and incentives

2. Calculate gross = basic + allowances + overtime + incentives

3. If gross <= 20000, deduction = 5% of gross

4. Else if gross <= 40000, deduction = 10% of gross

5. Otherwise, deduction = 15% of gross

6. Calculate net = gross - deduction

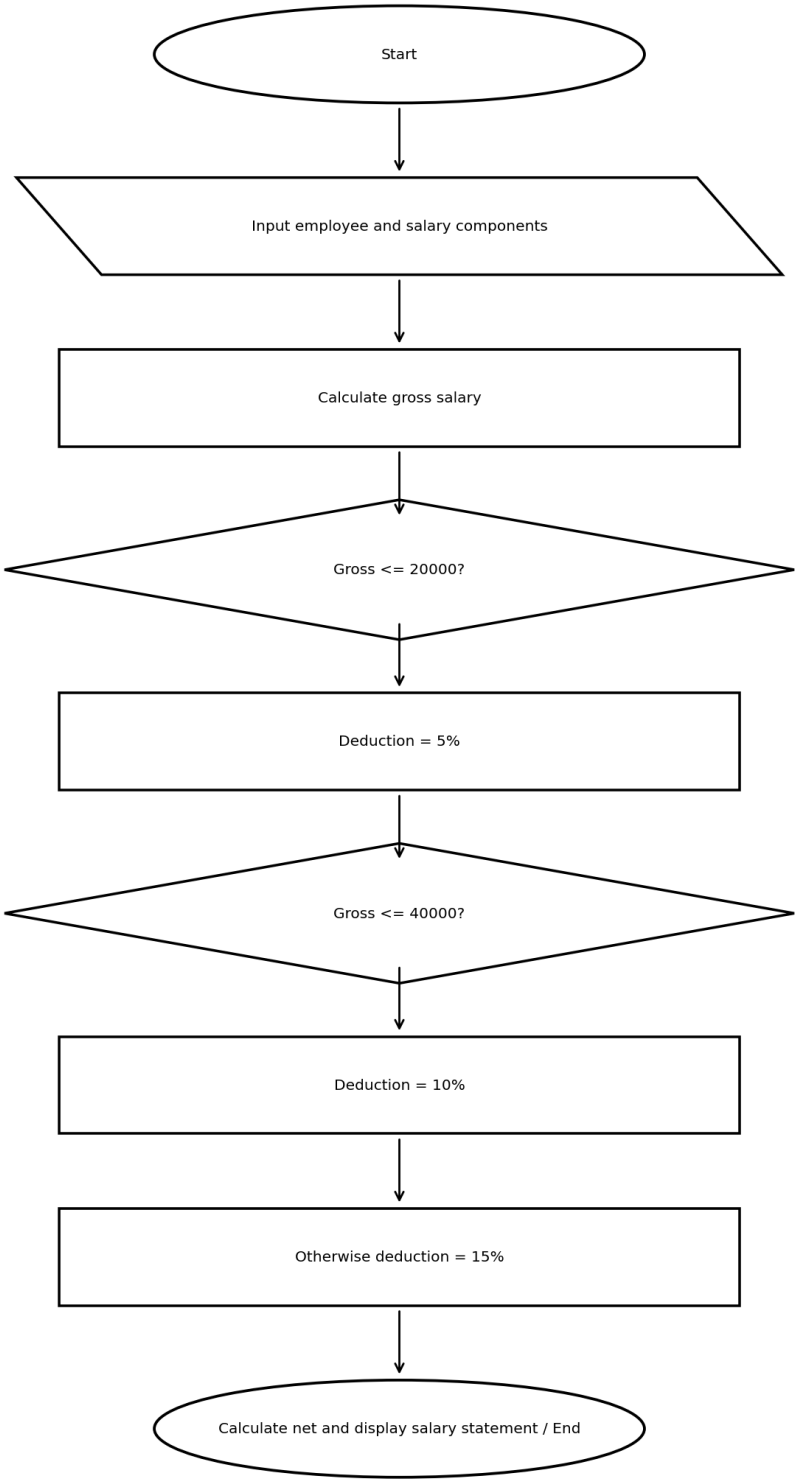
7. Display all salary details and net salary; End

Pseudocode

```
START
INPUT employee name, basic, allowances, overtime, incentives
gross = basic + allowances + overtime + incentives
IF gross <= 20000 THEN
    deduction = gross * 0.05
ELSE IF gross <= 40000 THEN
    deduction = gross * 0.10
ELSE
    deduction = gross * 0.15
END IF
net = gross - deduction
DISPLAY salary statement and net
STOP
```

Flowchart

Program 7: Employee Salary Statement



Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision

Python Source Code

```
name = input("Enter employee name: ")
basic = float(input("Enter basic salary: "))
allowances = float(input("Enter allowances: "))
overtime = float(input("Enter overtime pay: "))
incentives = float(input("Enter incentives: "))

gross = basic + allowances + overtime + incentives
if gross <= 20000:
```

```
    deduction = gross * 0.05
elif gross <= 40000:
    deduction = gross * 0.10
else:
    deduction = gross * 0.15

net = gross - deduction

print("\n--- Salary Statement ---")
print("Employee Name:", name)
print("Basic Salary:", basic)
print("Allowances:", allowances)
print("Overtime Pay:", overtime)
print("Incentives:", incentives)
print("Gross Salary:", gross)
print("Deduction:", deduction)
print("Net Salary:", net)
```

Sample Input

```
Karthik
25000
3000
2000
1000
```

Sample Output

```
Enter employee name: Karthik
Enter basic salary: 25000
Enter allowances: 3000
Enter overtime pay: 2000
Enter incentives: 1000
--- Salary Statement ---
Employee Name: Karthik
Basic Salary: 25000.0
Allowances: 3000.0
Overtime Pay: 2000.0
Incentives: 1000.0
Gross Salary: 31000.0
Deduction: 3100.0
Net Salary: 27900.0
```

Program 8: Campus Event Financial Planning

Aim

Campus Event Financial Planning

Objective

Calculate the total event cost, subtract sponsorship, estimate the minimum number of fee-paying students, and check whether actual contributions cover the budget.

Algorithm

1. Start; Input venue, equipment, catering, transport and certificate costs

2. Input sponsorship and registration fee per student

3. Calculate total event cost

4. Calculate remaining amount = total - sponsorship; if negative, set it to zero

5. If fee > 0, calculate ceiling(remaining / fee) as minimum students

6. Input actual participants and calculate participant contributions

7. If contributions + sponsorship < total, display shortfall; otherwise display budget covered

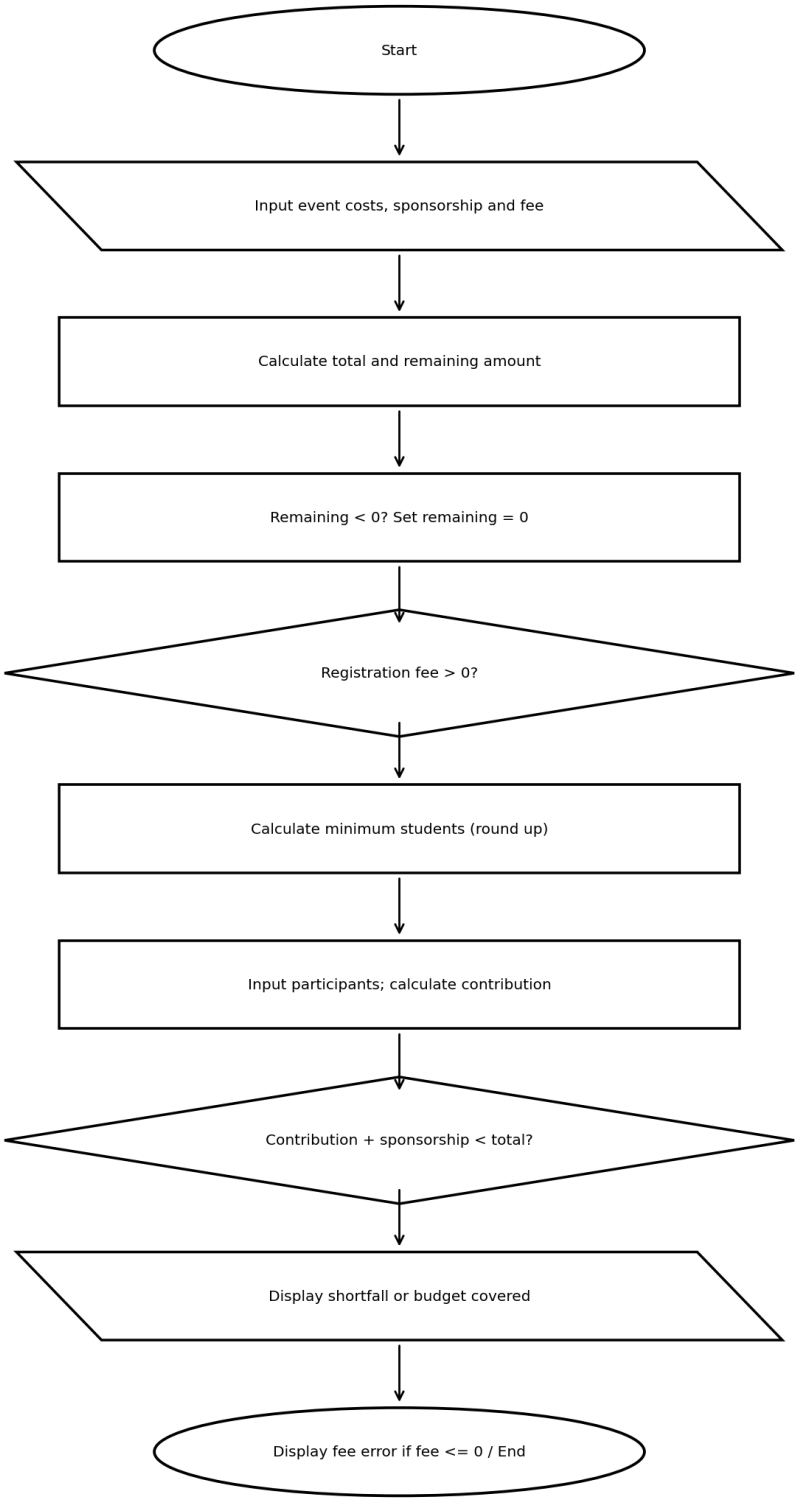
8. If fee <= 0, display fee validation message; End

Pseudocode

```
START
INPUT venue, equipment, catering, transport, certificates
INPUT sponsorship and fee
total = sum of all event costs
remaining = total - sponsorship
IF remaining < 0 THEN remaining = 0
DISPLAY total and remaining
IF fee > 0 THEN
    students = INTEGER(remaining / fee)
    IF students * fee < remaining THEN students = students + 1
    DISPLAY minimum students
    INPUT participants
    contribution = participants * fee
    DISPLAY contribution
    IF contribution + sponsorship < total THEN
        DISPLAY sponsorship shortfall
    ELSE
        DISPLAY "Event budget covered!"
    END IF
ELSE
    DISPLAY "Registration fee must be greater than zero."
END IF
STOP
```

Flowchart

Program 8: Campus Event Financial Planning



Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision

Python Source Code

```
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost: "))
transport = float(input("Enter transport cost: "))
certificates = float(input("Enter certificate cost: "))
sponsorship = float(input("Enter sponsorship amount: "))
fee = float(input("Enter registration fee per student: "))
total = venue + equipment + catering + transport + certificates
```

```
print("\nTotal Event Cost:", total)
remaining = total - sponsorship
if remaining < 0:
    remaining = 0
print("Amount needed after sponsorship:", remaining)

if fee > 0:
    students = int(remaining / fee)
    if students * fee < remaining:
        students += 1
    print("Minimum students needed:", students)
    participants = int(input("Enter actual participants: "))
    contribution = participants * fee
    print("Participant Contributions:", contribution)
    if contribution + sponsorship < total:
        print("Sponsorship shortfall:", total - contribution - sponsorship)
    else:
        print("Event budget covered!")
else:
    print("Registration fee must be greater than zero.")
```

Sample Input

```
10000
5000
12000
3000
1000
10000
100
250
```

Sample Output

```
Enter venue cost: 10000
Enter equipment cost: 5000
Enter catering cost: 12000
Enter transport cost: 3000
Enter certificate cost: 1000
Enter sponsorship amount: 10000
Enter registration fee per student: 100
Total Event Cost: 31000.0
Amount needed after sponsorship: 21000.0
Minimum students needed: 210
Enter actual participants: 250
Participant Contributions: 25000.0
Event budget covered!
```

Program 9: Bank Deposit and Withdrawal Transactions

Aim

Bank Deposit and Withdrawal Transactions

Objective

Process a specified number of deposits or withdrawals, validate transaction amounts and withdrawal limits, and display the final balance and successful transaction count.

Algorithm

1. Start; Initialize balance=5000, withdrawal limit=2000, transactions=0

2. Input number of transactions

3. Repeat for each transaction; display deposit and withdraw choices

4. Input choice and amount

5. If amount <= 0, display invalid amount

6. Else if choice=1, add amount to balance and increment successful transactions

7. Else if choice=2, check withdrawal limit, then available balance; subtract only if both checks pass

8. For any other choice, display invalid choice

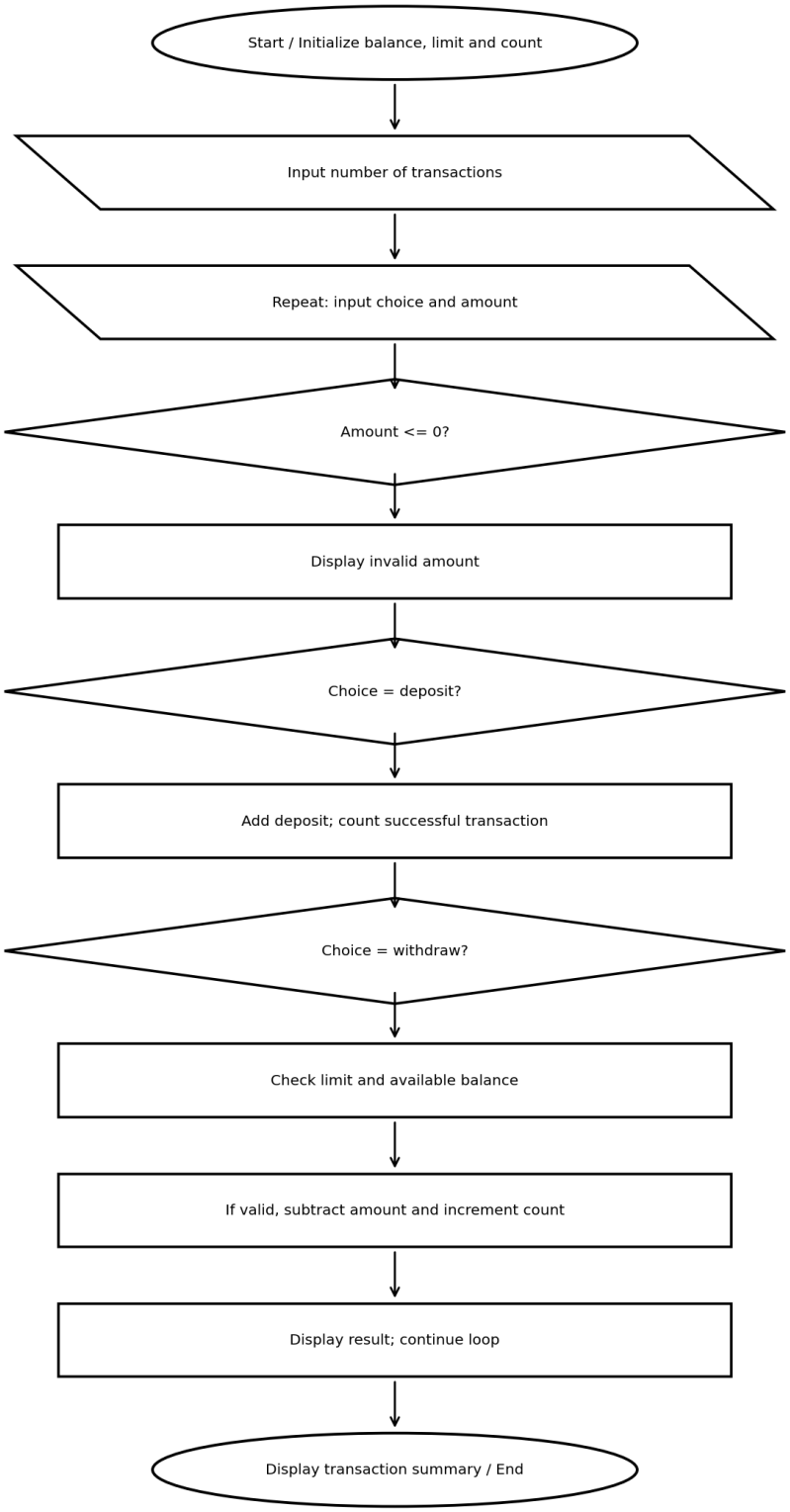
9. After loop, display successful transaction count and final balance; End

Pseudocode

```
START
SET balance=5000, limit=2000, transactions=0
INPUT n
FOR each transaction from 1 to n
    DISPLAY menu
    INPUT choice, amount
    IF amount <= 0 THEN
        DISPLAY invalid amount
    ELSE IF choice == 1 THEN
        balance = balance + amount
        transactions = transactions + 1
        DISPLAY deposit successful
    ELSE IF choice == 2 THEN
        IF amount > limit THEN DISPLAY withdrawal limit exceeded
        ELSE IF amount > balance THEN DISPLAY insufficient balance
        ELSE
            balance = balance - amount
            transactions = transactions + 1
            DISPLAY withdrawal successful
        END IF
    ELSE
        DISPLAY invalid choice
    END IF
END FOR
DISPLAY transactions and final balance
STOP
```

Flowchart

Program 9: Bank Deposit and Withdrawal Transactions



Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision

Python Source Code

```
balance = 5000
limit = 2000
transactions = 0
n = int(input("Enter number of transactions: "))

for i in range(n):
    print("1. Deposit")
    print("2. Withdraw")
```

```
    choice = int(input("Enter your choice: "))
    amount = float(input("Enter amount: "))

    if amount <= 0:
        print("Invalid amount!")
    elif choice == 1:
        balance += amount
        transactions += 1
        print("Deposit successful!")
    elif choice == 2:
        if amount > limit:
            print("Withdrawal limit exceeded!")
        elif amount > balance:
            print("Insufficient balance!")
        else:
            balance -= amount
            transactions += 1
            print("Withdrawal successful!")
    else:
        print("Invalid choice!")

print("\n--- Transaction Summary ---")
print("Total transactions:", transactions)
print("Final balance: Rs.", balance)
```

Sample Input

```
3
1
1000
2
1500
2
2500
```

Sample Output

```
Enter number of transactions: 3
1. Deposit
2. Withdraw
Enter your choice: 1
Enter amount: 1000
Deposit successful!
1. Deposit
2. Withdraw
Enter your choice: 2
Enter amount: 1500
Withdrawal successful!
1. Deposit
2. Withdraw
Enter your choice: 2
Enter amount: 2500
Withdrawal limit exceeded!
--- Transaction Summary ---
Total transactions: 2
Final balance: Rs. 4500.0
```

Program 10: Water Tank Level Monitoring for Five Days

Aim

Water Tank Level Monitoring for Five Days

Objective

Monitor tank water for up to five days, correct overflow or negative levels, and stop early when the water level reaches 800 units.

Algorithm

1. Start; Set capacity=1000, water=500, threshold=800, days=5

2. Repeat for each day from 1 to 5

3. Input inflow and outflow; update water level

4. If water exceeds 1000, display overflow and set it to 1000

5. If water is below 0, display shortage and set it to 0

6. Display current day and water level

7. If water >= 800, display threshold reached and exit loop

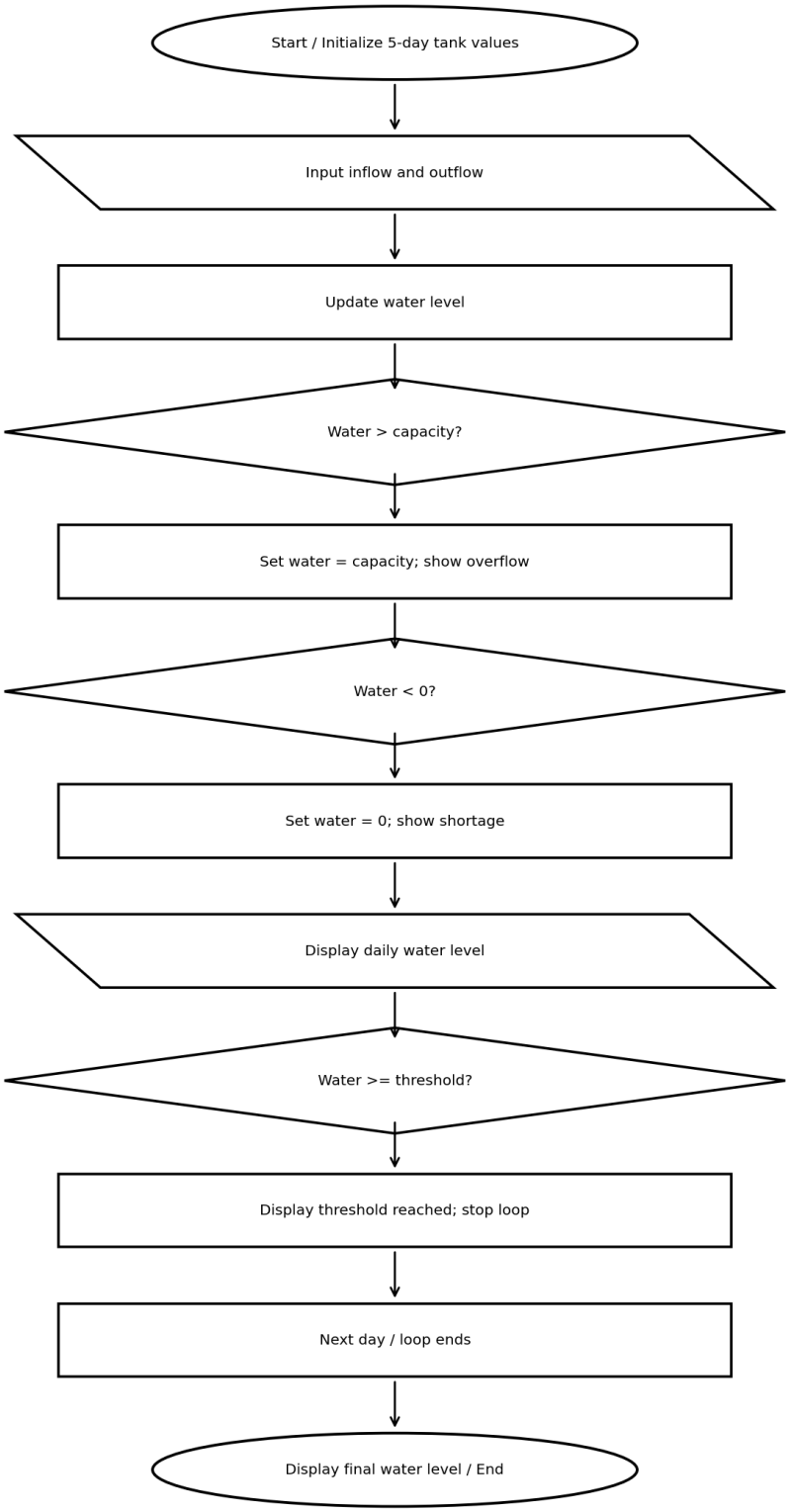
8. After loop, display final water level; End

Pseudocode

```
START
INITIALIZE capacity=1000, water=500, threshold=800, days=5
FOR day = 1 TO days
    INPUT inflow, outflow
    water = water + inflow - outflow
    IF water > capacity THEN
        DISPLAY overflow
        water = capacity
    END IF
    IF water < 0 THEN
        DISPLAY shortage
        water = 0
    END IF
    DISPLAY day and water
    IF water >= threshold THEN
        DISPLAY threshold reached
        EXIT LOOP
    END IF
END FOR
DISPLAY final water level
STOP
```

Flowchart

Program 10: Water Tank Level Monitoring for Five Days



Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision

Python Source Code

```
capacity = 1000
water = 500
threshold = 800
days = 5

for day in range(1, days + 1):
    inflow = int(input("Enter inflow: "))
    outflow = int(input("Enter outflow: "))
```

```
    water = water + inflow - outflow

    if water > capacity:
        print("Overflow!")
        water = capacity
    if water < 0:
        print("Water shortage!")
        water = 0

    print("Day", day, "Water level:", water)
    if water >= threshold:
        print("Threshold reached on day", day)
        break

print("Final water level:", water)
```

Sample Input

```
100
50
200
20
100
30
```

Sample Output

```
Enter inflow: 100
Enter outflow: 50
Day 1 Water level: 550
Enter inflow: 200
Enter outflow: 20
Day 2 Water level: 730
Enter inflow: 100
Enter outflow: 30
Day 3 Water level: 800
Threshold reached on day 3
Final water level: 800
```